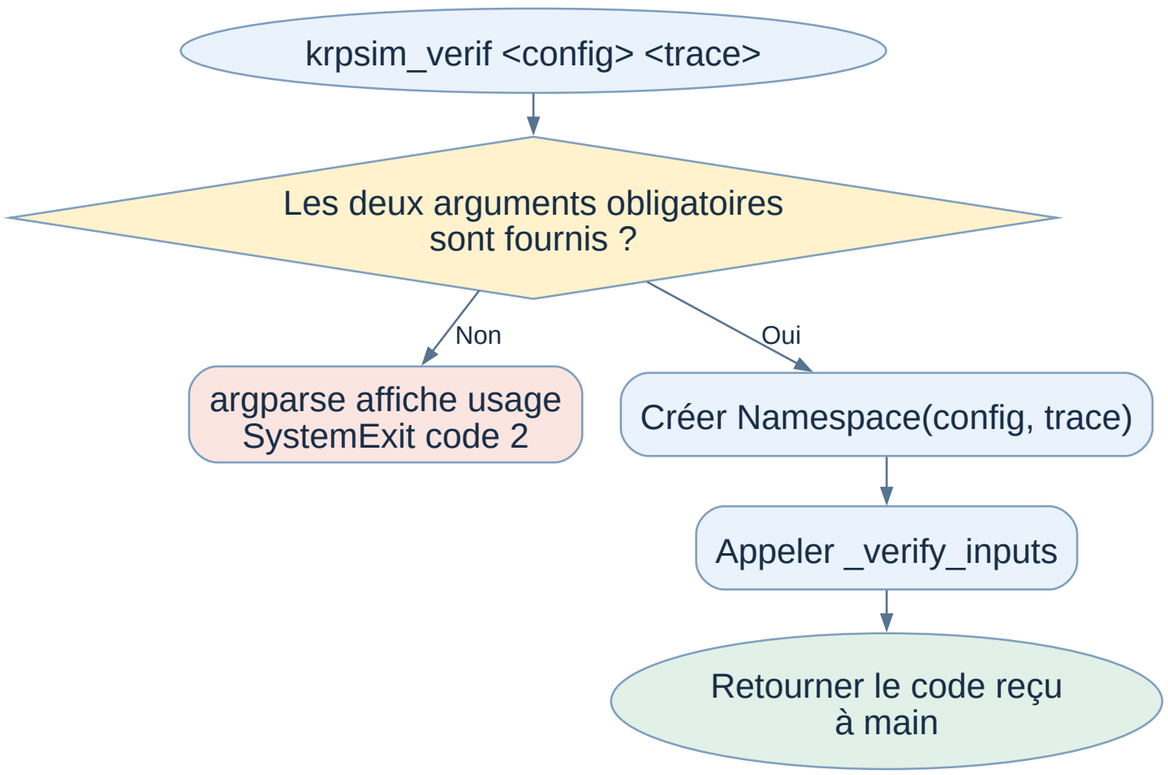
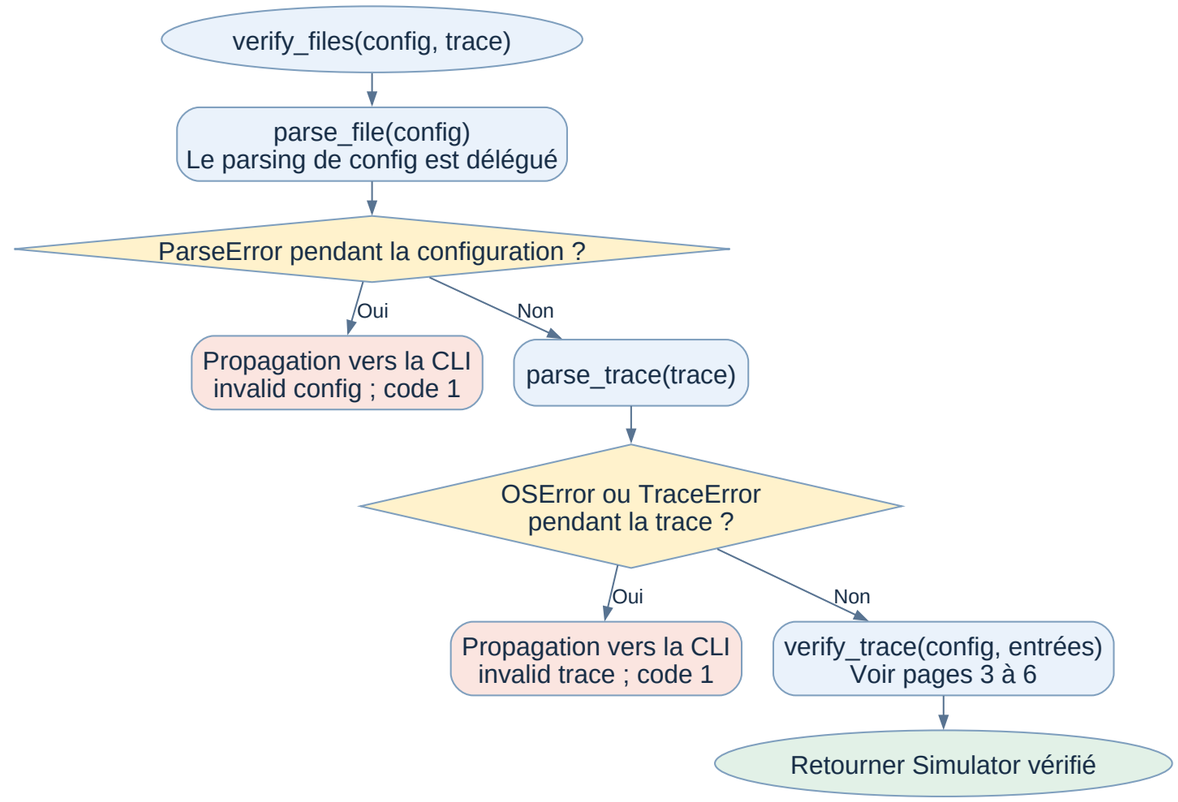
1. Entrer dans krpsim_verif

Commande, parsing des arguments, fichier de configuration, fichier de trace et diagnostics de la CLI

Interface argparse



Vérifier les deux fichiers dans l'ordre réel



Sources : src/krpsim_verif/cli.py:25-60, 75-102 ; verifier.py:253-280

Ordre de validation

La configuration est chargée avant le fichier de trace. Si elle est invalide, la lecture de la trace ne commence pas. verify_files ne convertit pas les exceptions : la CLI les transforme ensuite en messages.

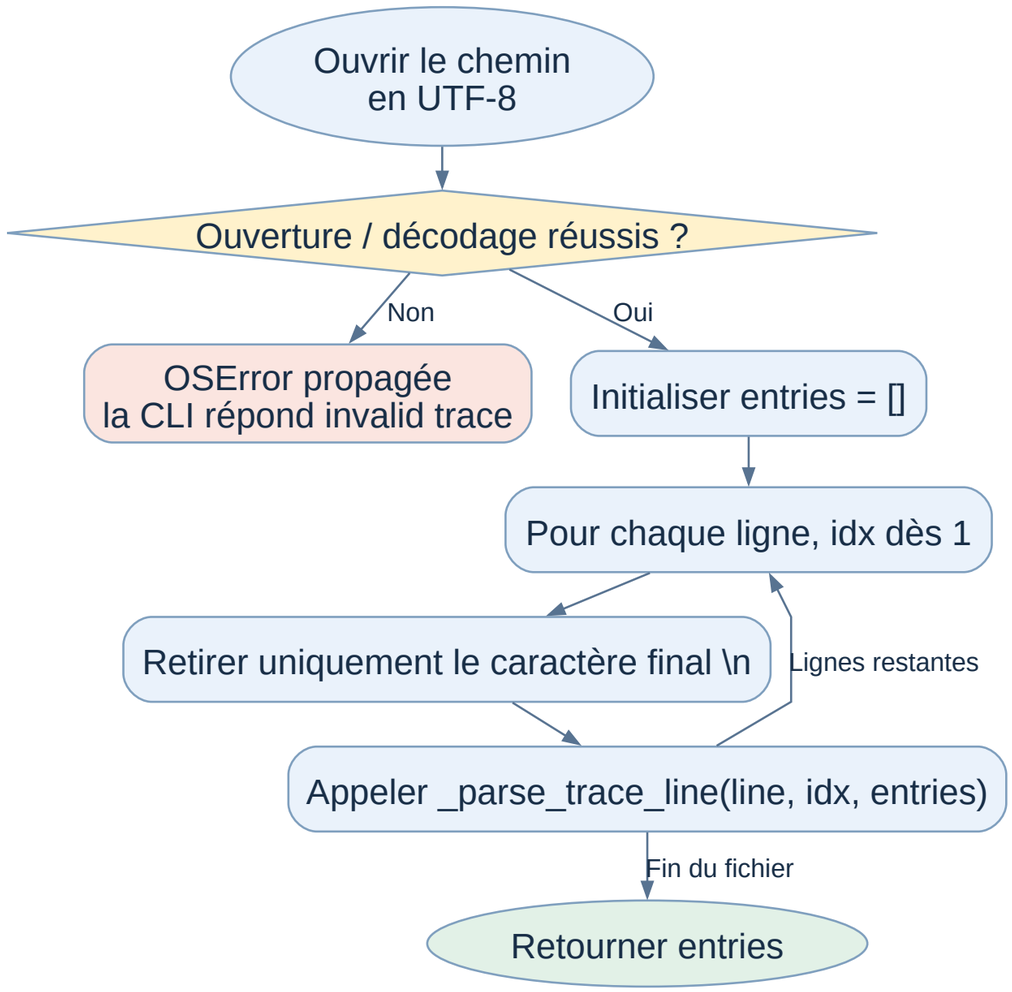
Exceptions interceptées par la CLI

ParseError devient « invalid config ». OSError et TraceError deviennent « invalid trace ». Les autres exceptions, par exemple une erreur technique inattendue, ne sont pas interceptées ici.

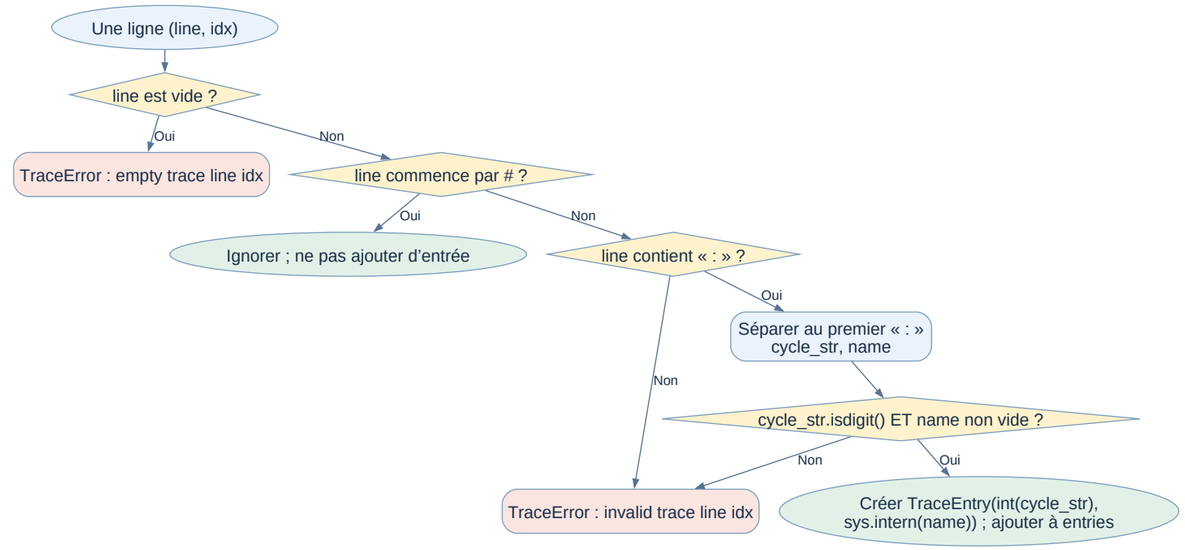
2. Lire le fichier de trace

parse_trace lit en UTF-8, numérote les lignes à partir de 1 puis délègue chaque ligne

Ouverture et parcours du fichier



Décisions exactes pour une ligne



Sources : src/krpsim_verif/verifier.py:63-110

Le contenu est conservé sans normalisation

Pas de strip des espaces. Seul « \n » final est retiré ; CRLF laisse donc le « \r ». Un commentaire est reconnu uniquement si « # » est le premier caractère. Une ligne vide est une erreur, tandis qu'un fichier vide produit une liste vide.

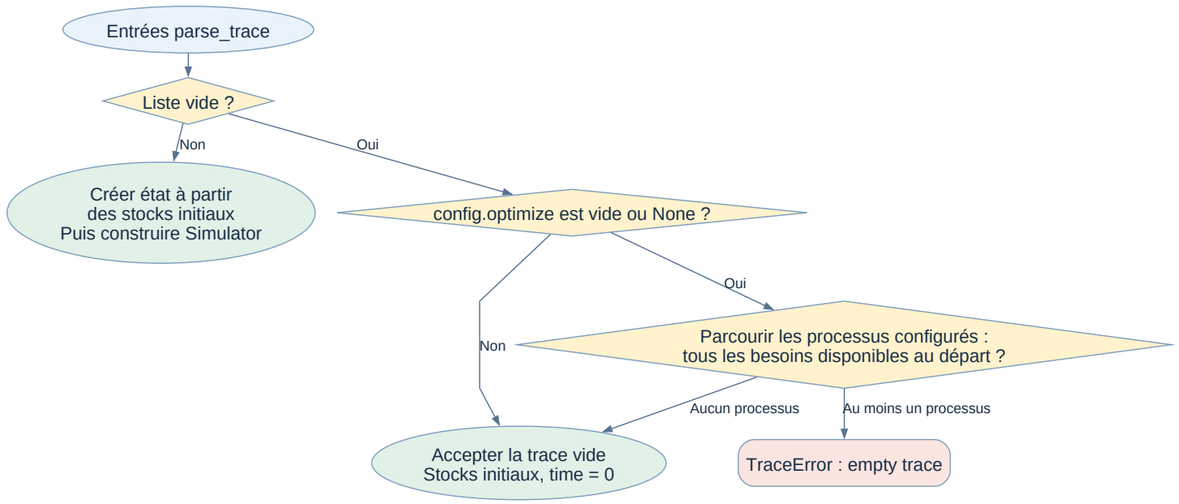
Format accepté

Cycle : chaîne où str.isdigit() est vrai. Séparateur : premier « : ». Nom : non vide ; les autres « : » dans le nom restent permis. Les zéros initiaux sont convertis par int. Une ValueError imprévue de int n'est pas convertie en TraceError.

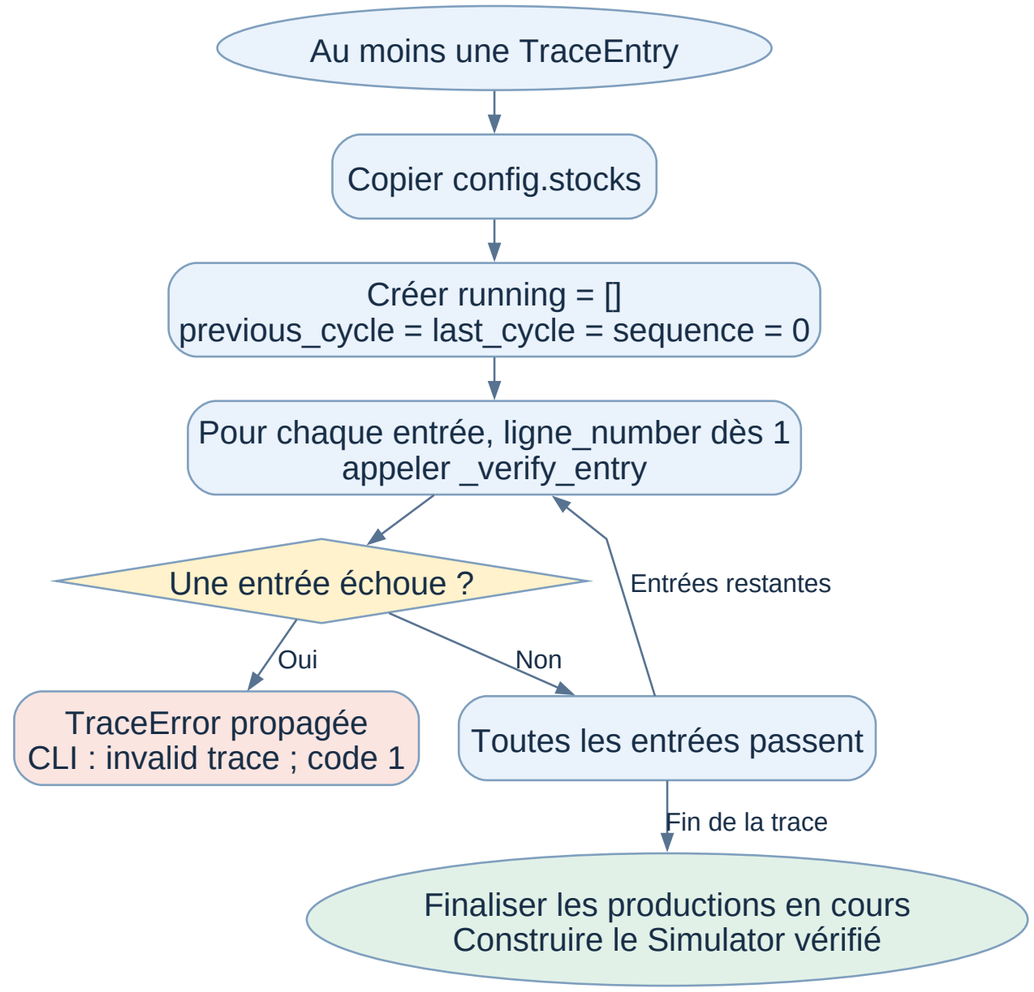
3. Choisir la voie pour rejouer la trace

verify_trace distingue la trace vide du parcours des entrées ; l’algorithme ne recalcule pas la trace optimale

Cas particulier : zéro entrée



Cas général : initialiser puis traiter chaque événement



Sources : src/krpsim_verif/verifier.py:52-59, 113-120, 218-249

Quand une trace vide est-elle acceptée ?

Sans optimize, elle est rejetée si au moins un processus peut démarrer à partir des stocks initiaux. Les durées et les résultats ne sont pas pris en compte dans ce test. Avec toute directive optimize non vide, la trace vide est acceptée.

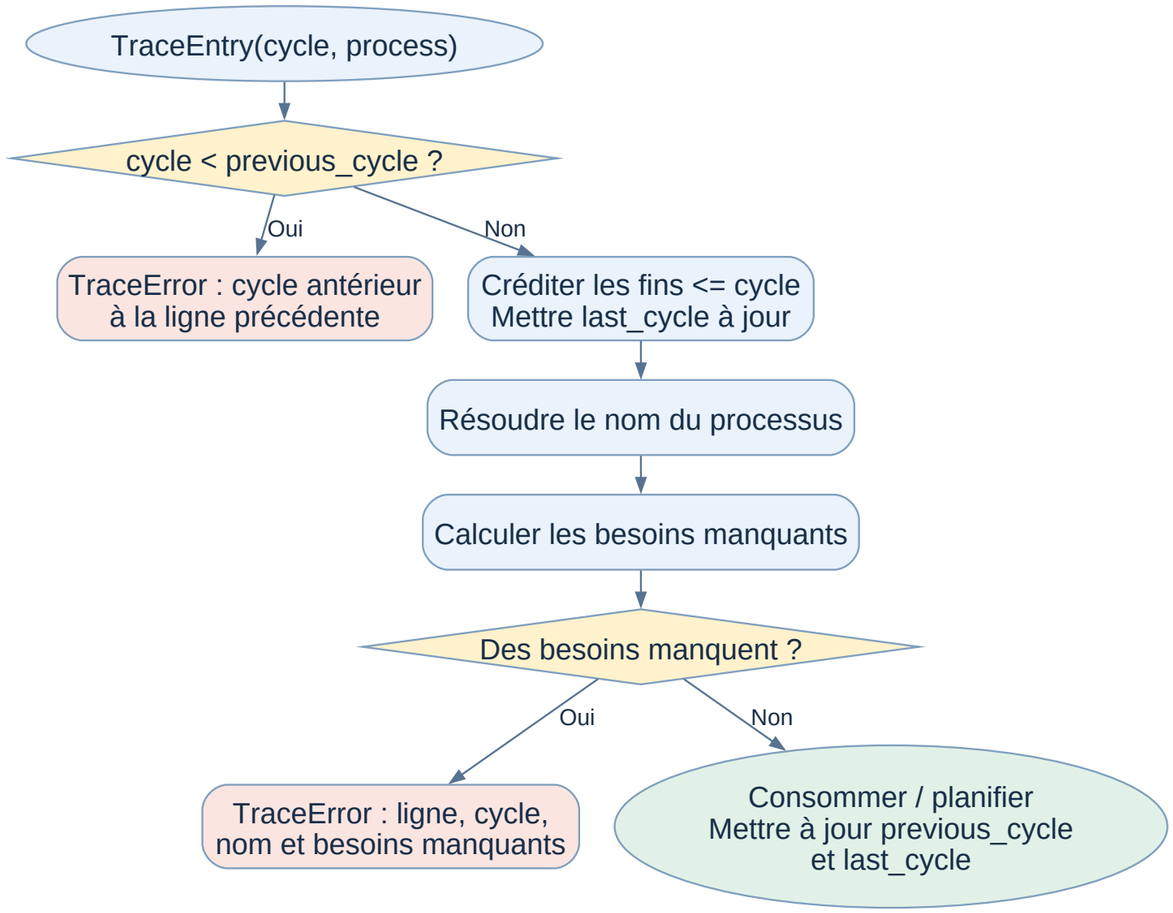
Trace partielle acceptée

Une trace non vide n’est pas comparée à l’ordonnancement de krpsim et n’a pas à terminer le but. Le vérificateur rejoue uniquement les événements fournis, valide leurs ressources et crédite les processus encore en cours à la fin.

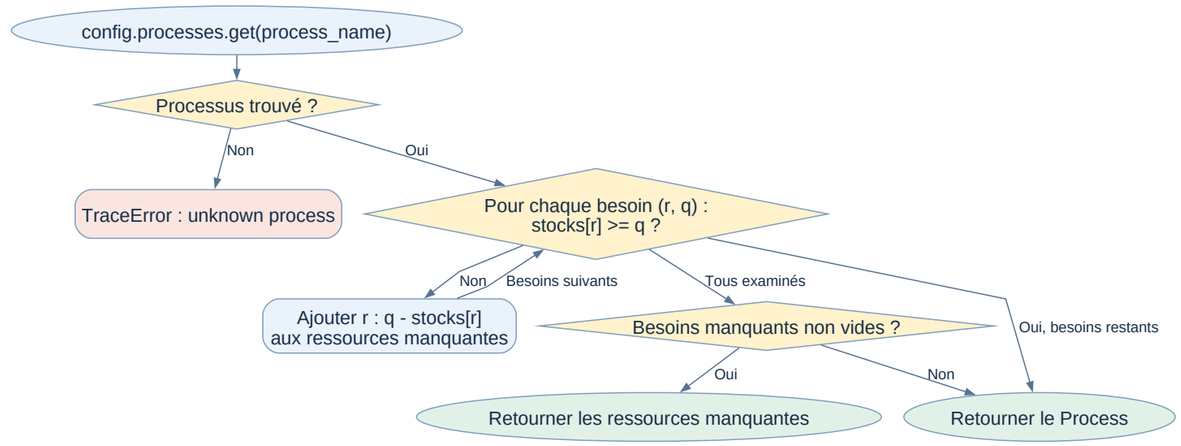
4. Valider chaque départ

Ordre strict dans la liste ; plusieurs départs peuvent partager un cycle

_verify_entry : chronologie puis complétions



Après les complétions : processus et ressources



Sources : src/krpsim_verif/verifier.py:123-155, 178-202

Égalité du cycle acceptée

Un cycle égal au précédent est valide. Le premier événement doit être au moins au cycle 0 car previous_cycle commence à 0 ; le parseur de trace exige aussi un cycle numérique.

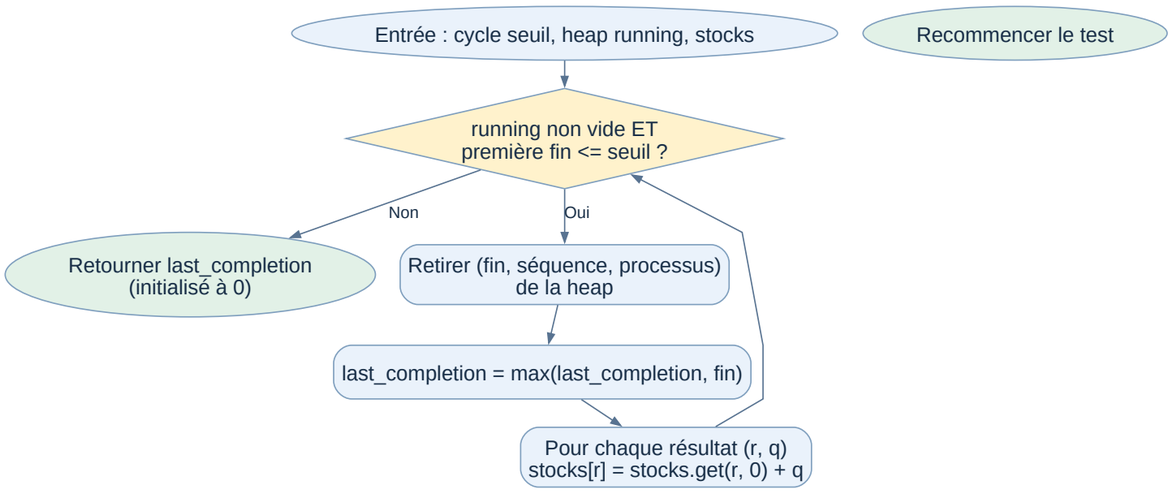
Ordre du traitement

Les processus qui finissent au cycle de départ sont crédités avant de vérifier ses besoins. La première erreur interrompt le rejeu. La validation ne compare ni le nom du processus à un plan choisi, ni le cycle au délai de krpsim.

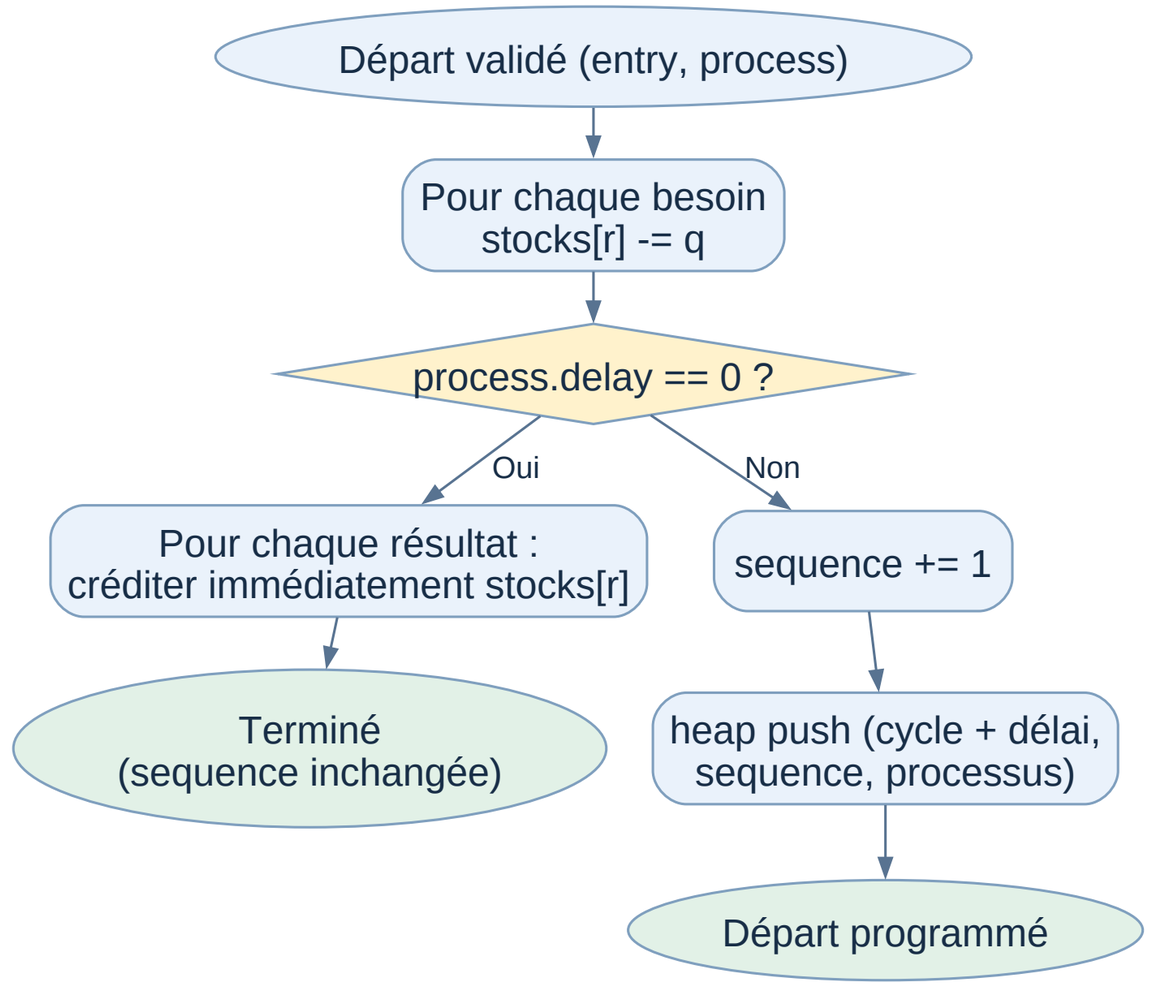
5. Créditer les résultats au bon moment

Le rejeu consomme les besoins à chaque départ et crédite les résultats soit immédiatement, soit à la fin du délai

_complete_until : vider la file arrivée à échéance



_consume_and_schedule : appliquer un départ valide



Sources : src/krpsim_verif/verifier.py:52-59, 123-136, 158-175

Pourquoi la séquence est dans le tas ?

La heap trie d’abord par cycle de fin, puis par sequence. Ce compteur croissant donne un ordre stable aux processus qui finissent ensemble et évite de comparer leurs objets Process.

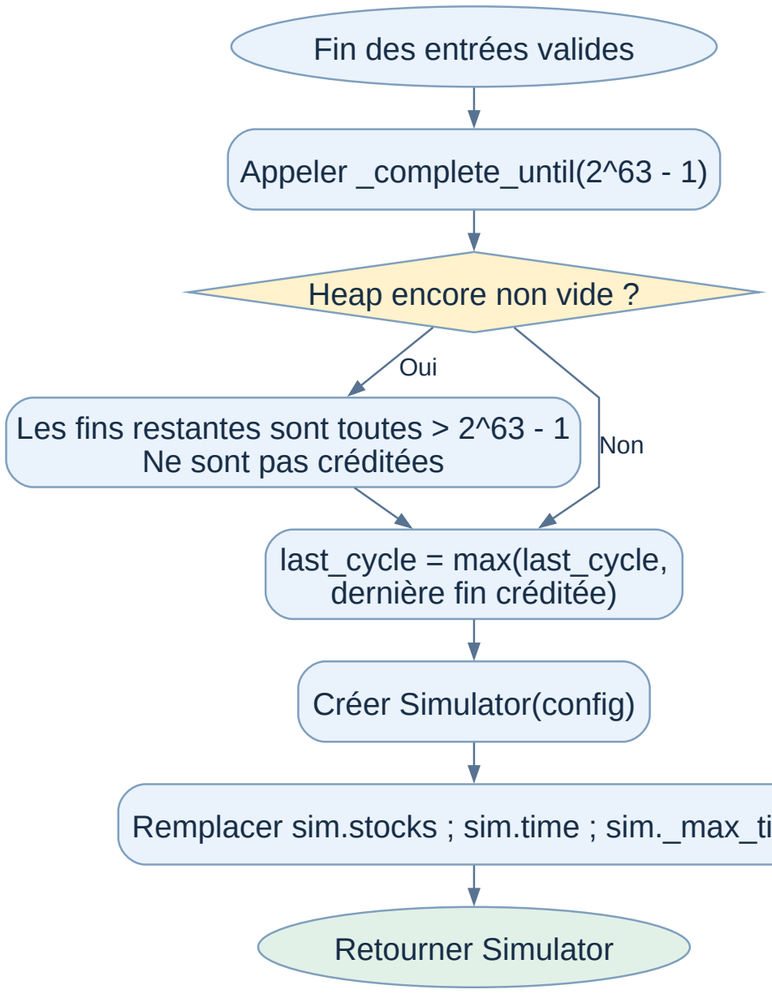
Stocks de la même ressource

Chaque résultat est ajouté à la valeur courante avec défaut zéro. Les besoins sont déduits avant toute programmation. Un processus de durée nulle produit ses résultats au même cycle, visibles par l’entrée suivante du même cycle.

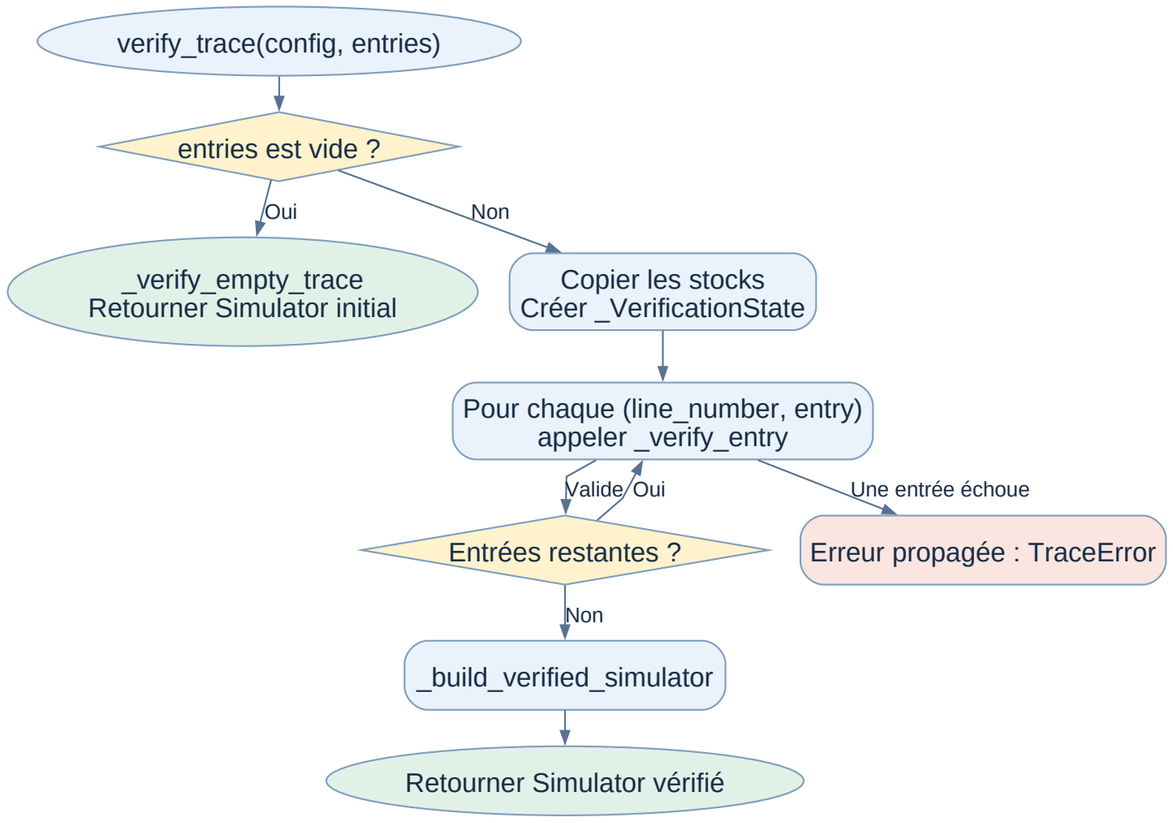
6. Finaliser l’état vérifié

Construire le résultat après chaque événement, puis créditer le travail qui finit après le dernier départ

_build_verified_simulator : fin de rejeu



verify_trace : orchestrer les deux voies



Sources : src/krpsim_verif/verifier.py:123-136, 205-249

Plafond de finalisation

Le code crédite les fins jusqu’à 2^63 - 1. Un cycle de départ ou une durée permettant une fin supérieure à ce plafond reste en cours et ses résultats ne sont pas ajoutés. Le dernier cycle affiché est le maximum des cycles d’entrée et fins créditées.

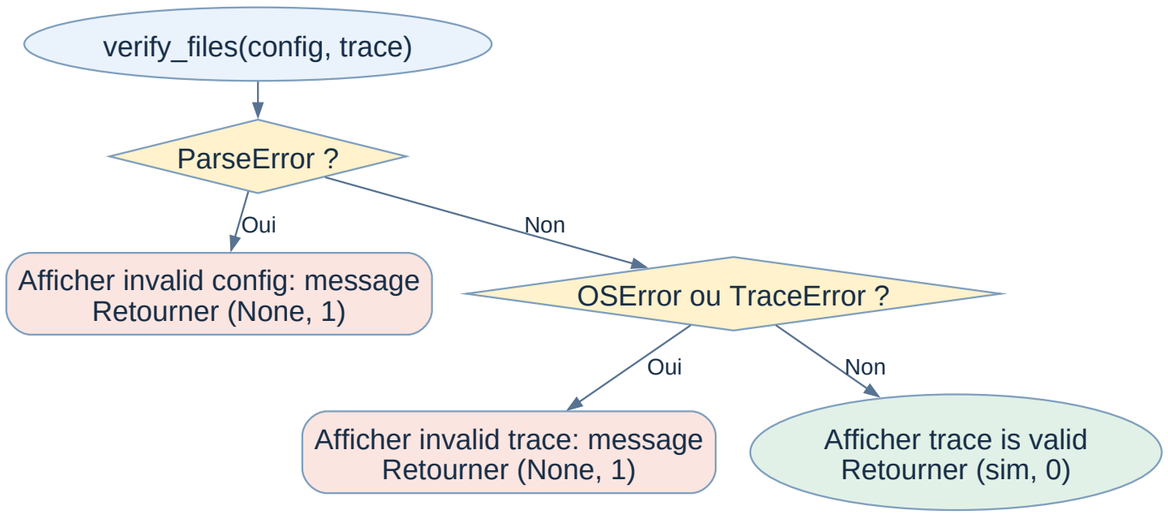
État retourné

Les stocks retournés sont ceux du rejeu. sim.time reçoit last_cycle et sim._max_time reçoit aussi last_cycle. Le vérificateur accepte une trace valide plus courte que la simulation complète.

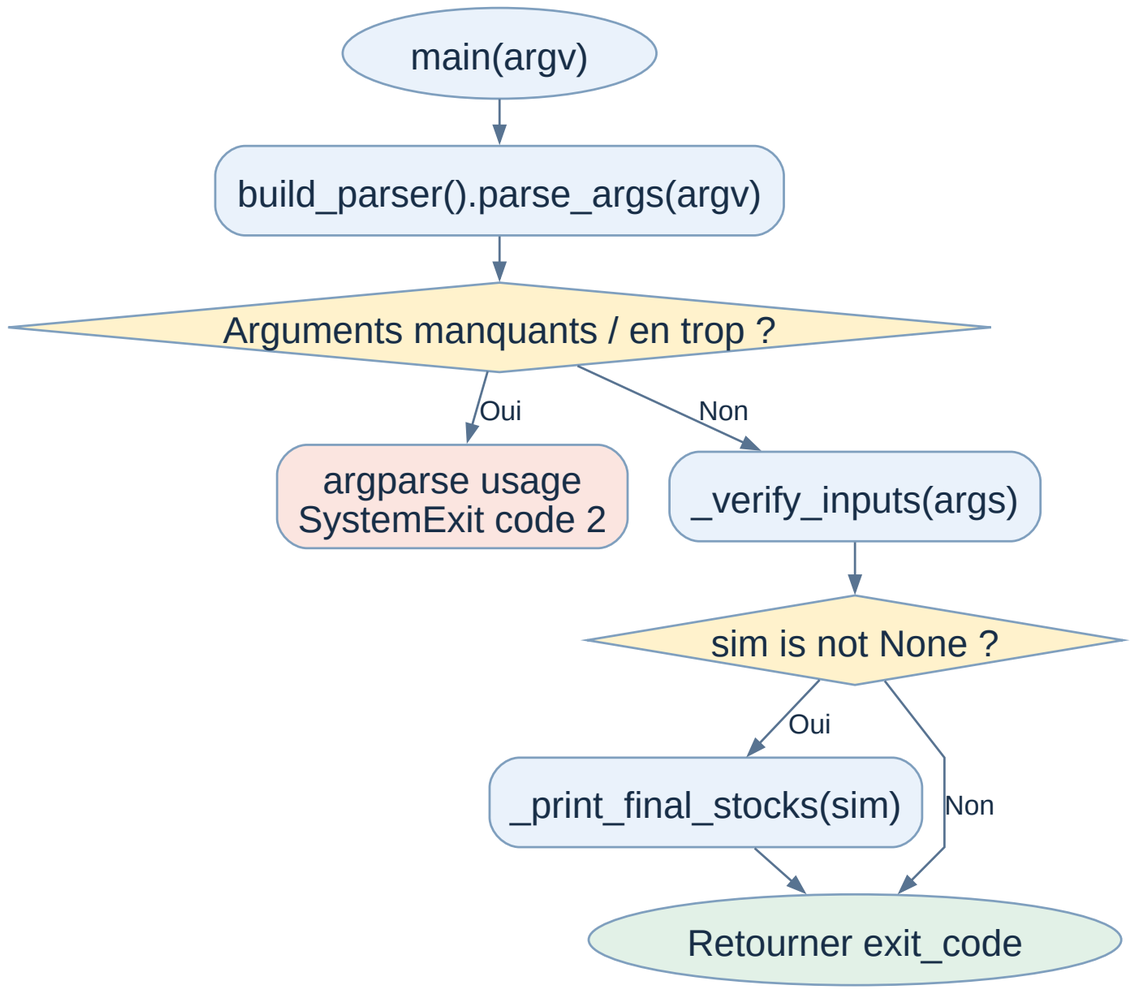
7. Retourner le diagnostic de la CLI

Messages, état final et codes de sortie ; les erreurs argparse précèdent le rejeu

_verify_inputs : convertir les erreurs attendues



main : afficher les stocks seulement après succès



Sources : src/krpsim_verif/cli.py:25-102

Affichage des stocks

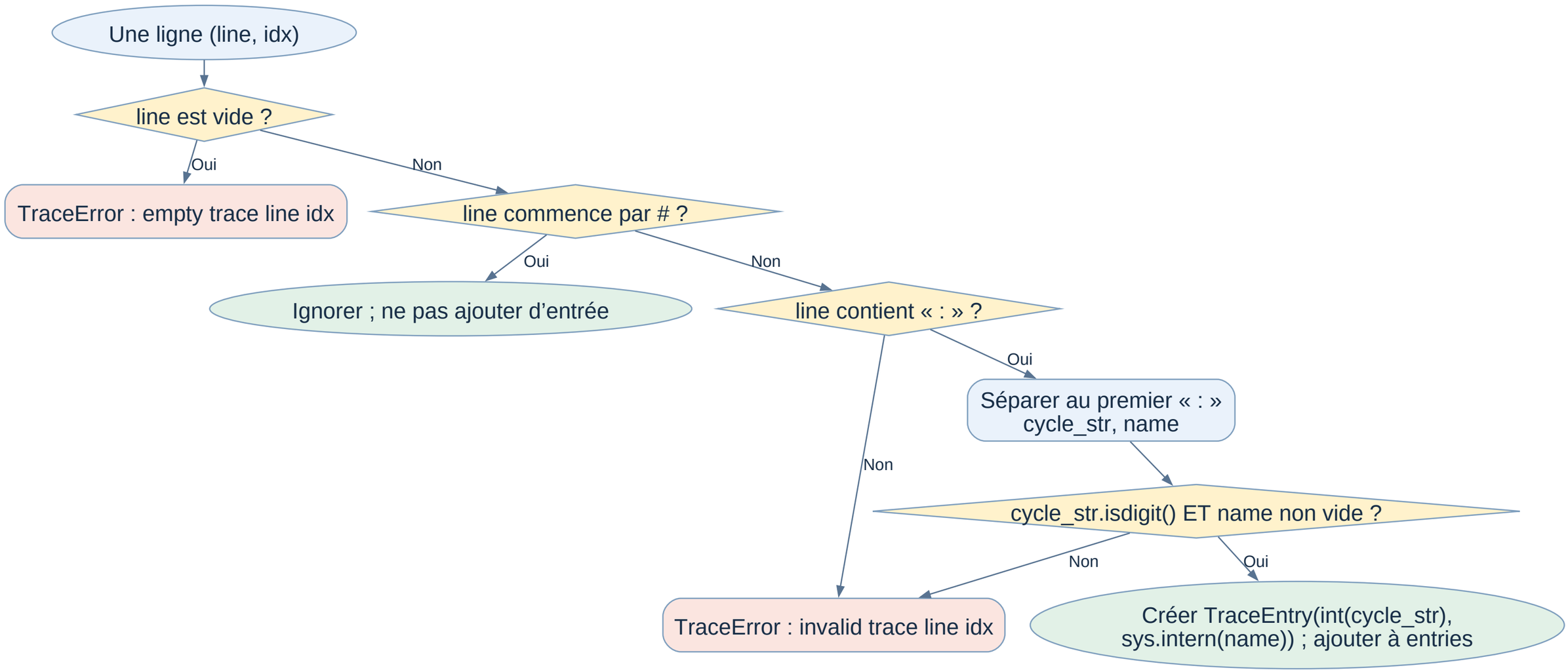
Tri alphabétique des noms de ressources. Largeur alignée sur le nom le plus long. Chaque ressource affiche sim.stocks.get(name, 0) ; Last cycle affiche sim.time. Ces lignes ne sont imprimées que si la vérification réussit.

Ce que le code 0 certifie

Format et ordre des événements acceptés, processus connu, besoins disponibles, stocks rejoués, fins restantes créditées jusqu'au plafond du code. Cela ne certifie ni l'optimalité, ni que la trace soit complète, ni le respect du délai demandé à krpsim.

8. Analyser une ligne de trace

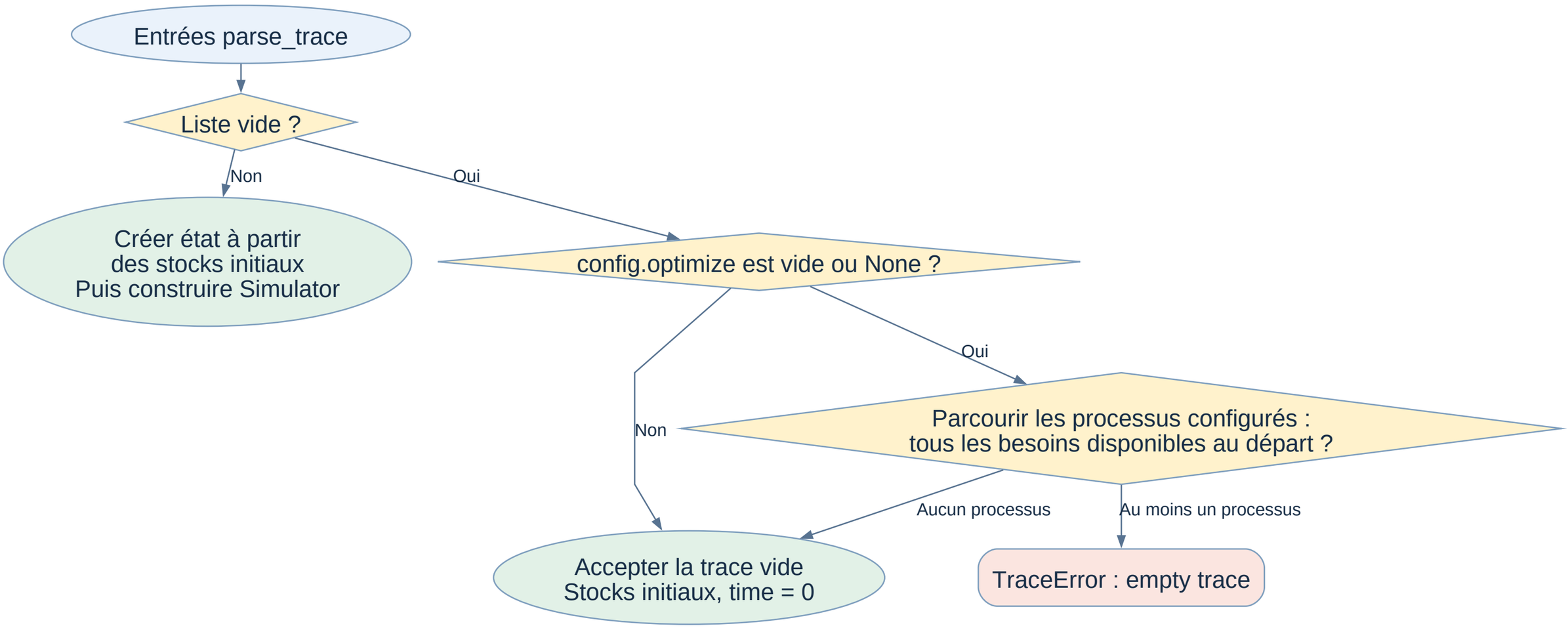
Détail exhaustif de _parse_trace_line et de ses erreurs



Source : src/krpsim_verif/verifier.py:85-110

9. Décider si une trace vide est valide

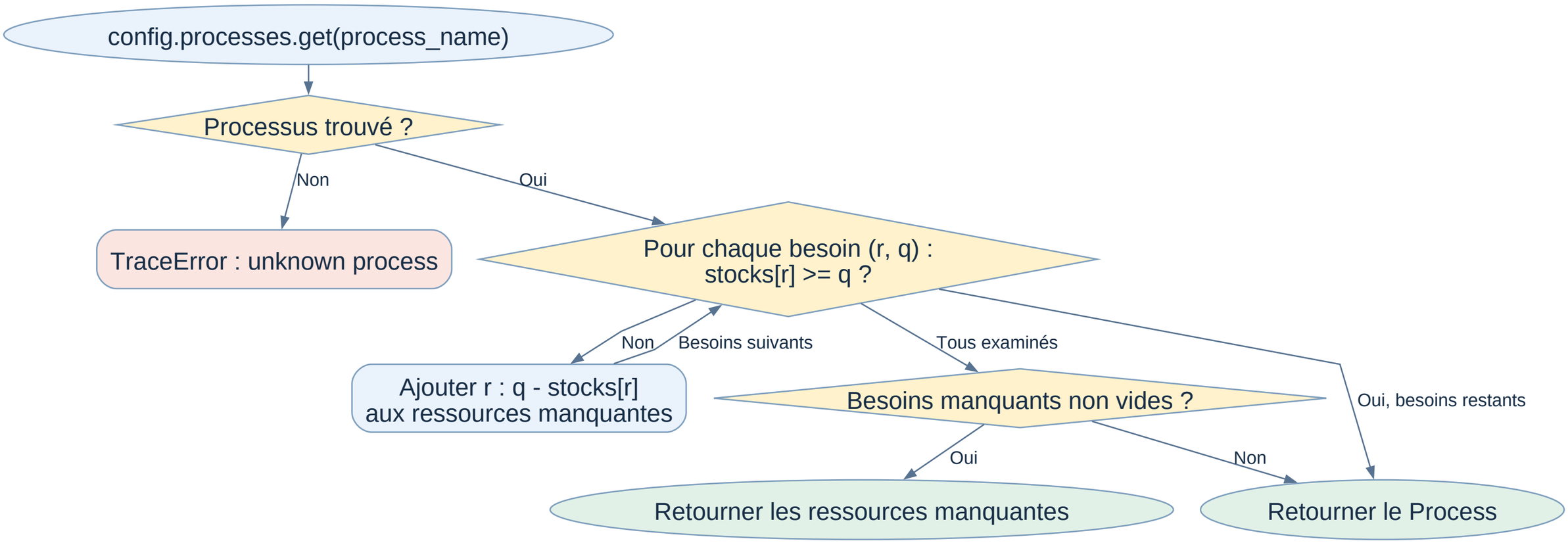
Disponibilité initiale des besoins et directive optimize



Source : src/krpsim_verif/verifier.py:52-59, 113-120

10. Résoudre le processus et ses besoins

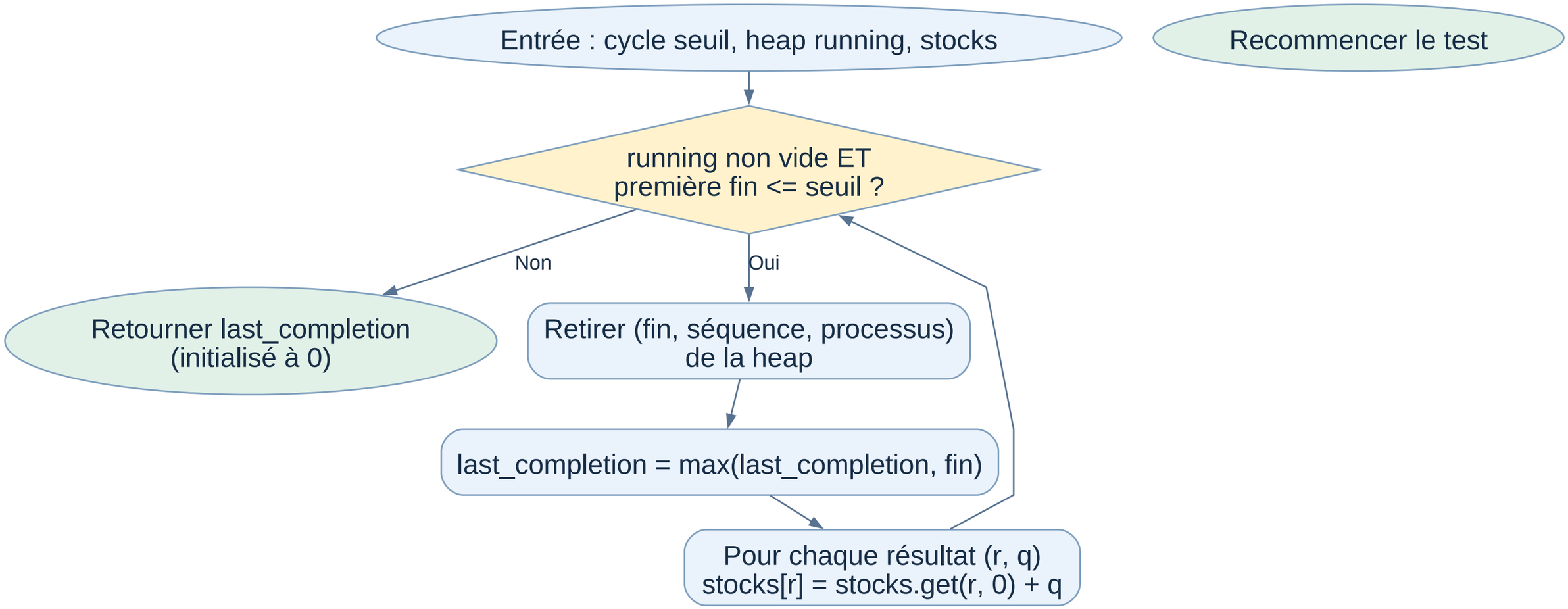
Recherche du nom puis contrôle indépendant de chaque ressource requise



Source : src/krpsim_verif/verifier.py:138-155

11. Créditer les processus terminés

Ordre des fins, mise à jour des stocks et arrêt au seuil demandé



Source : src/krpsim_verif/verifier.py:158-175